

Lab 4 — Programming with a DBMS (Python + SQLite → PostgreSQL)

Module I · Week 4 · Duration: 2 hours Starter: starter/app.py · Deliverable: completed app.py + README.md

Objectives

Turn SQL knowledge into a working application. You will:

1. Use the Python **DB-API** (`sqlite3`, then `psycopg2`) to talk to a database.
2. Write **parameterized** CRUD operations (and understand why).
3. Implement a **transaction** that is atomic — commit on success, **roll back** on failure.
4. **Demonstrate** an SQL-injection attack and then **prevent** it.
5. **Repoint** the same app from SQLite to **PostgreSQL** with minimal change.

Time budget (approx. 2 h): setup + read scaffold ~20 min · Part 1 (CRUD + `return_book`) ~30 min · Part 2 (transactions) ~20 min · Part 3 (injection) ~25 min · Part 4 (PostgreSQL) ~25 min.

Part 0 — Window Functions & Views (from Lab 3)

Window Functions

15. **Rank products within each category by price** (`RANK() OVER (PARTITION BY cat_id ORDER BY price DESC)`); return the **top 2 per category** (wrap the window query in a CTE and filter `rank <= 2`).
16. **Running total** of order counts by month (`COUNT(*)` grouped by month in a CTE, then `SUM(...) OVER (ORDER BY month)`).
17. For each customer: total spend, their **rank by spend**, and spend as a **percentage of the top spender** (`MAX(total) OVER ()`).

Warning: You **cannot** filter on a window column (`rank`) in the same query's `WHERE`. Compute it in a CTE/subquery, then filter in the outer query.

Views

19. Create a view:

```
CREATE VIEW product_sales AS
SELECT p.prod_id, p.prod_name,
       COALESCE(SUM(oi.quantity), 0) AS units,
       COALESCE(SUM(oi.quantity * oi.unit_price), 0) AS revenue
FROM   products p
LEFT JOIN order_items oi ON oi.prod_id = p.prod_id
GROUP BY p.prod_id, p.prod_name;
```

20. Query the view: the **5 best-selling products by revenue**.
21. Query the view again: products with `units = 0` (never sold).

Checkpoint: Checkpoint: `SELECT COUNT(*) FROM product_sales;` equals the number of products (~23) — the `LEFT JOIN` keeps unsold products.

Short Answer (Window Functions)

- **B.** For Q15, explain why the window function had to be wrapped in a CTE

Stretch Goals (Window Functions)

- Add `LAG` to Q16 to show month-over-month **change** in order counts.
- Use `NTILE(4)` to split customers into spend quartiles.

Setup (approx. 20 min)

```
bash cd starter python3 app.py --selftest          # should print several PASS lines, exit
0 python3 app.py                                   # interactive menu on library.db
```

The scaffold is a small **Library catalog**:

```
books(book_id, title, author, copies) members(member_id, name) loans(loan_id, book_id,
member_id, returned)
```

Read app.py top to bottom. Note especially:

- `connect()` — chooses SQLite/PostgreSQL and the placeholder (`?` vs `%s`). - `add_book`, `search_books` — **parameterized** queries. - `search_books_unsafe` — an intentionally vulnerable version (Part 3). - `borrow_book` — a **transaction** (decrement copies + insert loan atomically). - `return_book` — a `NotImplementedError` you will complete.

> **Checkpoint: Checkpoint:** `--selftest` prints `RESULT: ALL PASS`.

Part 1 — Complete the CRUD (approx. 30 min)

1.1 Implement `return_book(conn, book_id, member_id)`

Mirror `borrow_book`, but in reverse, **inside one transaction**:

1. Find an **outstanding** loan (`returned = 0`) for that book+member. If none exists, raise a `ValueError`.
2. Mark that loan `returned = 1`.
3. **Increment** the book's `copies` by 1.
4. `commit()` on success; on any error `rollback()` and report.

Use **parameterized** statements (`{PH}`) throughout.

1.2 Add a “list members / list loans” menu option

Add menu item 7) `List outstanding loans` that prints, for each un-returned loan: member name, book title. (Requires a join — you wrote plenty in Lab 2.)

Checkpoint: Checkpoint: borrow a book, then return it, and confirm `copies` returns to its original value and the loan shows `returned = 1`.

Part 2 — Transactions & atomicity (approx. 20 min)

2.1 Study `borrow_book`. In your `README.md`, answer:

- Which two writes must happen **together**? What inconsistency occurs if only one happens?
- Where exactly does the rollback occur when copies are 0?

2.2 **Prove atomicity.** Borrow book 4 (`copies = 0`). Confirm via the menu (or a query) that:

- the operation **fails**,
- **no** new `loans` row was created, and
- book 4's `copies` is still 0.

2.3 Add a **deliberate** bug: between the UPDATE and the INSERT in a copy of `borrow_book`, raise `RuntimeError("boom")`. Run it and show that, thanks to the rollback, `copies` is **not** left decremented. Then remove the bug. Paste the before/after `copies` value into `README.md`.

Checkpoint: Checkpoint: the injected failure leaves the database exactly as it was before the call (atomicity in action).

Part 3 — SQL injection: attack, then defend (approx. 25 min)

3.1 **Attack.** Run the app, choose **6) Search books (UNSAFE)**, and enter:

```
%' OR '1'='1' --
```

Observe that it returns **every** book — the WHERE was subverted. Then try:

```
%' UNION SELECT member_id, name, name FROM members --
```

and observe that it **leaks the members table** through the book-search screen.

3.2 In `README.md`, explain in 2–3 sentences **why** this works: how the input became part of the SQL command.

3.3 **Defend.** Change menu option **6** to call the **safe search_books** (parameterized) instead of **search_books_unsafe**. Re-run both attack strings and confirm they now return **no** rows (the input is treated as a literal search term). Delete or clearly quarantine **search_books_unsafe** with a comment.

3.4 **Rule to write in your README:** *“Never build SQL by string concatenation; always pass user input through placeholders.”*

Checkpoint: Checkpoint: after the fix, the attack strings return 0 rows; normal searches (e.g. Database) still work.

Part 4 — Repoint to PostgreSQL (approx. 25 min)

The same app should run on a real server DBMS with minimal change.

4.1 Create the database and role (see `infrastructure.md`), then:

```
createdb -U csl303 library          # or via psql / Docker
```

4.2 Run against PostgreSQL:

```
python app.py --backend postgres --selftest
```

- The `connect()` function already switches the placeholder to `%s` and imports `psycopg`. **Adjust the connection string** (host/user/password/dbname) in `connect()` for your setup (marked with a `TODO`).
- If `--selftest` passes on PostgreSQL too, you’ve achieved **portability**: one codebase, two engines.

4.3 In `README.md`, list **every** change you had to make to move from SQLite to PostgreSQL. (Hint: it should be small — placeholders, the connection string, and possibly `lastrowid` vs `RETURNING` if you used it.)

Checkpoint: Checkpoint: `python app.py --backend postgres --selftest` prints `RESULT: ALL PASS`.

Stretch goals (optional)

- Replace hard-coded `book_ids` with **auto-generated** keys (`INTEGER PRIMARY KEY / RETURNING book_id`) and update the code accordingly.
- Add a **retry loop** around `borrow_book` for PostgreSQL `SERIALIZABLE` conflicts.

- Add input validation and friendly error messages for every menu action.
-

Submission

Submit:

1. **app.py** — with `return_book` implemented, menu option 7 added, option 6 switched to the safe search, and the PostgreSQL connection filled in.
2. **README.md** — answers for Parts 2.1, 3.2, and 4.3, plus the atomicity before/after values, and the one-line injection rule.

Self-check before submitting:

```
python app.py --selftest           # SQLite: ALL PASS
python app.py --backend postgres --selftest # PostgreSQL: ALL PASS
```

Both must pass, and the two injection attack strings must return **no** rows through menu option 6.